Demand Prediction

Worcester Polytechnic Institute

Golf Course Demand Prediction

Neel Awsare, Pedro Nogueira, Calvin Lambert

MA4635: Data Analytics & Statistical Learning

Professor Oren Mangoubi

05/01/2026

Introduction

Golf courses have historically used simplified forms of dynamic pricing, adjusting rates according to the time of day or day of the week. Typically, this is done through fixed pricing for mornings, afternoons, weekdays, and weekends, with weekend mornings being priced the highest and weekday afternoons the lowest. While this pricing structure reflects general demand, it stays static and fails to account for variation in demand driven by weather conditions, seasonality, and booking behavior. According to a case study on Missouri Bluffs Golf Club, which implemented a dynamic pricing system in 2022, there is untapped revenue that can be accessed through an advanced pricing structure. Missouri Bluffs Golf Course has experienced a 36.3% increase in total green fee revenue over a two-year period, alongside a 44.7% increase in online bookings. These results suggest that dynamic pricing can improve revenue and fill out tee sheets. However, such a system heavily depends on the ability of the course to accurately anticipate demand.

This project aims to develop and test statistical learning models to predict demand using weather conditions, calendar features, and crowdedness. We seek to identify the primary drivers of demand and evaluate which modeling approach, linear regression, lasso regression, random forest, or gradient boosting, produces the most accurate predictions. These predictions will give a baseline for implementing a dynamic pricing strategy that will draw more customers on lower demand days and maximize profit on higher demand days.

Dataset Description

The dataset used in this project is the *Golf Play Dataset Extended*, available on Kaggle and created by Samy Baladram. It extends the classic "Golf Play" dataset by incorporating additional variables and a larger number of observations to better reflect realistic golfing and weather conditions. Although the data is synthetically generated, it is designed to resemble real-world patterns in Sendai, Japan, where golf can be played year-round.

The dataset is structured in a wide format, where each row represents a single day from December 31, 2020, to December 30, 2023, resulting in 1,095 observations. It includes temporal features (weekday, month, season), weather-related variables (temperature, humidity, wind conditions, and outlook), and outcome variables related to golf activity.

This project's target variable is crowdedness, a continuous value between 0 and 1 that measures how busy the course is on a given day. Since the dataset does not explicitly state the calculation this variable, we define crowdedness as the proportion of tee times filled relative to the total number of tee times available:

$$Crowdedness = \frac{Tee\ Times\ Filled}{Tee\ Times\ Offered}$$

This formula provides a measure of demand that allows comparisons across days. Player-level variables (individual player participation and duration) were excluded from the analysis, as the objective is to model aggregate course demand rather than individual behaviors. Additionally, date was deleted from the dataset, removing time-series correlations from the project scope.

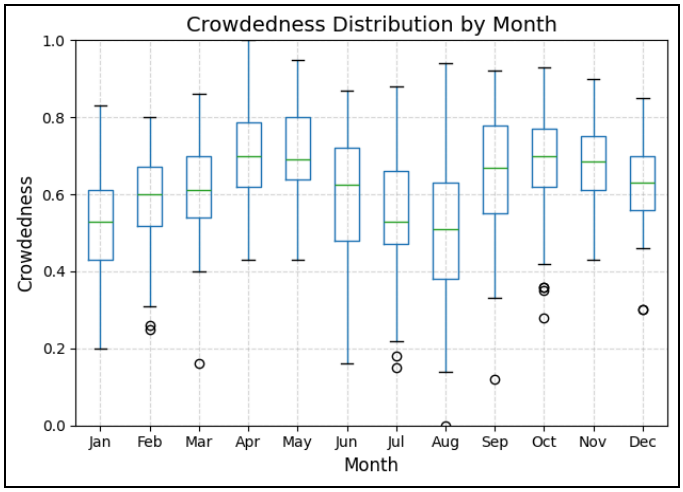
After removing player-related and non-informative variables, the dataset is reduced to nine features used for modeling, defined as follows:

1. **Weekday:** Ordinal from 0 to 6, weekdays Sunday through Saturday.
2. **Month:** Ordinal from 0-11, calendar months January through December.
3. **Season:** Categorical, representing Winter, Spring, Summer, or Autumn.
4. **Holiday:** Binary, whether the day is a holiday or not.
5. **Temperature:** Continuous numerical variable, measuring daily temperature (assumed to represent the daily high, not explicitly specified).
6. **Humidity:** Continuous numerical variable, measuring as relative humidity for the day.
7. **Windy:** Binary, whether there are windy conditions or not.
8. **Outlook:** Categorical, representing Sunny, Overcast, Snow, or Rain.
9. **Crowdedness:** Continuous numerical variable, ranging from 0.0 to 1.0, representing overall course demand.

Preliminary Analysis

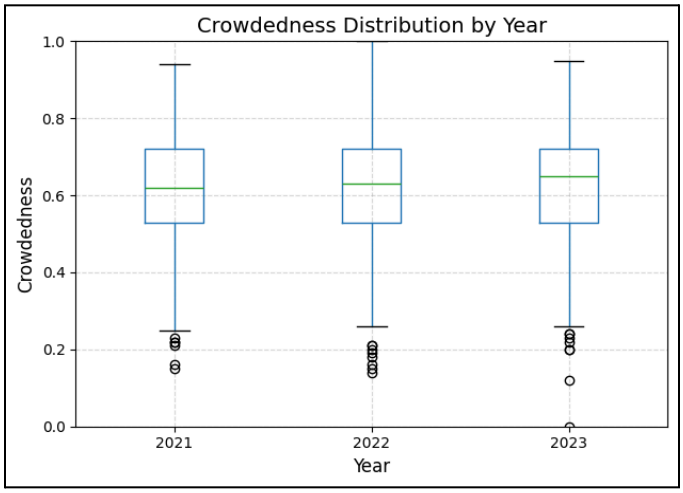
Having established the initial set of features, we conducted exploratory data analysis to examine underlying patterns in the dataset. The goal of this analysis is to understand how years, seasons, and weekdays influence crowdedness, prior to modeling. The following questions were investigated:

<u>What patterns exist in crowdedness across seasons?</u>



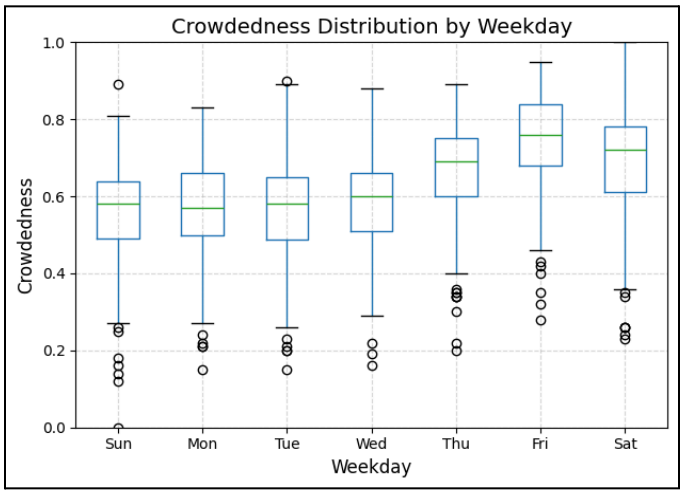
The boxplots show clear seasonal variation in crowdedness across months, with higher demand observed in spring and fall. Median demand peaks around April to May and again in September to November. In contrast, lower median demand is observed in winter and late summer months, particularly January and August. Overall, this pattern suggests that encoding month as a cyclical variable may be effective for capturing seasonal trends in the modeling process.

<u>How does crowdedness vary across years?</u>



The boxplots show that crowdedness is consistent from 2021 to 2023, with similar medians and variability each year. There is a slight upward shift in median demand over time, suggesting a modest increase in average crowdedness. All years exhibit a similar spread, along with a few lower-end outliers representing occasional low-demand days. Overall, yearly variation appears minimal, indicating that other factors likely play a larger role in driving demand.

How does crowdedness vary across weekdays?



The boxplots show a clear weekly pattern in crowdedness, with demand increasing as the week progresses. Median crowdedness is stable from Sunday through Wednesday, with similar distributions and variability. On Thursday, demand rises noticeably, with Friday and Saturday showing the highest median values and overall levels of crowdedness. Golf course demand is higher toward the end of the week, indicating that weekday effects are an important factor and may be effectively captured using cyclical encoding in the modeling process.

Overall, the exploratory data analysis indicates that crowdedness follows clear cyclical patterns across both weekday and month, with minimal difference across years. To effectively capture these trends, weekday and month will be encoded using cyclical transformations during preprocessing.

Preprocessing

The dataset used in this study is synthetically generated and does not contain recording errors. As a result, no observations were removed during the cleaning process. A check for missing values confirmed that no variables contained NA entries. Additionally, outliers were retained in the dataset, as these values are not the result of measurement error but instead represent valid, but rare, simulated conditions. Preserving these observations ensures that the full distribution of the data is maintained for modeling purposes.

Based on patterns identified during exploratory data analysis, temporal features were transformed to better capture cyclical behavior. Specifically, the weekday was encoded using sine and cosine transformations to represent its periodic nature:

$$Weekday_{sin} = sin(\frac{2\pi \cdot Weekday}{7}),\ Weekday_{cos} = cos(\frac{2\pi \cdot Weekday}{7})$$

The same approach was applied to the month variable to capture seasonal trends. Additionally, temperature values were converted from Celsius to Fahrenheit to help with interpretability.

The dataset was divided into training and testing sets using a 90%–10% split. The training set will be used for model development and evaluation through five-fold cross-validation, allowing for robust estimation of model performance. The test set will be held out entirely during training and used only for final model evaluation to assess generalization performance.

Model Evaluation and Comparison

Linear Regression

Linear regression will serve as the baseline model, estimating crowdedness as a weighted sum of all input features, with no regularization. All features are included and each has a coefficient estimated by ordinary least squares.

Metric	**Value**
Train R^2	0.6593
Test R^2	0.6208
Test RMSE	0.1003
Cross-Validated R^2	0.645 ± 0.011

The close difference between training and test accuracy indicates that the model is not overfitting, the gap is small and the cross-validation scores are stable. The test RMSE of 0.1003 means predictions are off by about 10% in crowdedness on average. While this provides a useful baseline, the result of 0.62 leaves substantial unexplained variance, most likely suggesting that the true relationships are nonlinear.

Lasso Regression

Lasso regression extends ordinary least squares by adding a penalty on the coefficient vector to the loss function: the model minimizes RSS + $\alpha \cdot \Sigma|\beta_i|$, where α controls the strength of regularization. The penalty has the geometric property that it drives some coefficients to near or exactly to zero, which is great for feature selection. This makes Lasso useful both for improving prediction in high-noise settings and for identifying which predictors contribute meaningfully to the outcome.

Because Lasso penalizes coefficients by their raw magnitude, features must be standardized before fitting; otherwise variables with larger scales would be penalized disproportionately. Each feature was centered and scaled to unit variance using `StandardScaler`, fit on the training set and applied to the test set.

To select the regularization strength α, the model was evaluated across a grid of candidate values:

α	Test R^2
0.0001	0.6205
0.001	0.6179
0.01	0.5530
0.1	-0.0002
1.0	-0.0002

The performance drop at $\alpha = 0.01$ and the collapse beyond that, indicates that most predictors are significant contributors to the outcome. The best result was achieved at $\alpha = 0.0001$ (the weakest penalty in the grid) which produced a test R^2 of 0.6205, nearly identical to linear regression.

Feature	Coefficient
Humidity	-0.0783
Weekday_sin	-0.0595
Weekday_cos	-0.0088

Month_sin	+0.0089
Month_cos	-0.0084
Temperature	+0.0335
Windy	-0.0275
Holiday	+0.0242
Outlook_sunny	+0.0240
Outlook_rainy	-0.0196
Outlook_snow	-0.0431
Outlook_overcast	**0.0000**

Because these are standardized coefficients, magnitudes are directly comparable across features. Humidity is the dominant predictor; higher humidity is associated with lower crowdedness. Notably, Month is among the smallest non-zero coefficients, an order of magnitude below the other coefficients. The only feature driven exactly to zero is Outlook_overcast, indicating that overcast conditions add no predictive value once the other outlook categories and continuous weather variables are accounted for.

A key preprocessing decision was how to represent the time of year. Two encodings were considered: Month, transformed into a cyclic pair, and Season, encoded as four one-hot dummies. Both were evaluated by fitting Lasso under identical conditions and comparing test R^2 and cross-validation stability. Season produced higher test R^2 and tighter cross-validation variance.

However, despite Lasso's preference for Season, Month was retained for the final pipeline. Re-running the random forest and gradient boosting models under both encodings produced statistically insignificant prediction differences. Month was preferred on this resolution: a 12-level encoding produces predictions tied to a specific month rather than a 3-month seasonal bucket, which is more useful for pricing decisions that operate on monthly or finer timescales. The Lasso disadvantage is accepted as a tradeoff isolated to the linear models.

Random Forest

Random forest is an ensemble method that builds a large number of decision trees, and averages their predictions. Each tree is trained on a bootstrap sample of the training data, and at each split, only a random subset of features is considered as candidates; this is a technique called feature bagging. This is a model that can capture complex nonlinear relationships and feature interactions that a single linear model cannot do. The final model used 300 trees, no depth limit, and a minimum of 2 samples at any leaf.

Metric	**Value**
Train R^2	0.9607
Test R^2	0.8757
Cross-Validated R^2	0.849 ± 0.008

The gap between training and test indicates some degree of overfitting. However, the test still represents a substantial improvement over the linear models, demonstrating that the true relationships in the data are nonlinear and that the ensemble captures them effectively. The cross-validation confirms that the model generalizes reliably.

Feature	**Importance**
Humidity	0.384
Weekday_sin	0.219
Weekday_cos	0.013
Month_sin	0.009
Month_cos	0.009
Temperature	0.200
Windy	0.027
Holiday	0.035

Outlook_sunny	0.037
Outlook_rainy	0.014
Outlook_snow	0.050
Outlook_overcast	0.004

Three features account for most of the total feature importance; humidity, weekday, and temperature.

Gradient Boosting

Gradient boosting is a sequential ensemble method that builds trees one at a time, where each new tree is trained to correct the residuals left by the ensemble before. Unlike random forest, which reduces variance by averaging independent trees, gradient boosting reduces both bias and variance by fitting to the negative gradient of the loss function.

The final model used 500 trees, a learning rate of 0.05, a maximum depth of 3 per tree, a minimum of 5 samples per leaf, and a subsampling of 80% of the training data at each iteration.

Metric	**Value**
Train R^2	0.9425
Test R^2	0.8978
Cross-Validated R^2	0.874 ± 0.014

Gradient boosting achieves the best test R^2 of all four models at 0.898, outperforming random forest on the test set. The gap between train and test is also smaller than random forest, suggesting the shallower tree depth and subsampling provide effective regularization. The cross-validated mean of 0.874 further confirms strong generalization on the test set.

Feature	**Importance**
Humidity	0.353
Weekday_sin	0.218

Weekday_cos	0.005
Month_sin	0.004
Month_cos	0.004
Temperature	0.189
Windy	0.033
Holiday	0.039
Outlook_sunny	0.076
Outlook_rainy	0.010
Outlook_snow	0.054
Outlook_overcast	0.013

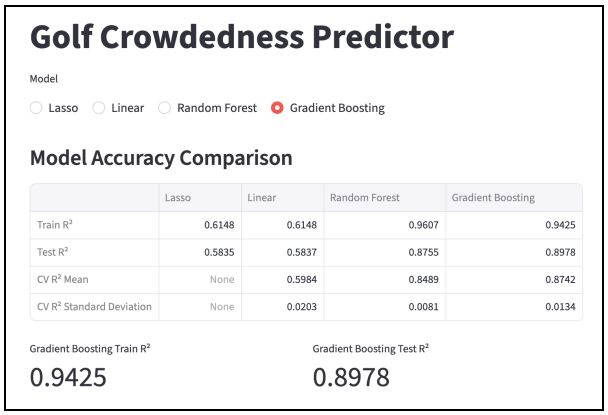
This reinforces that humidity, weekday, and temperature are the primary drivers of demand.

Model	**Train R^2**	**Test R^2**	**Cross-Validated R^2**
Linear Regression	0.615	0.584	0.598 ± 0.020
Lasso (α = 0.0001)	0.615	0.584	0.598 ± 0.020
Random Forest	0.961	0.876	0.849 ± 0.008
Gradient Boosting	0.943	0.898	0.874 ± 0.014

The linear and lasso models R^2 plateau around 0.58, which establishes the ceiling for purely additive relationships. Both tree-based methods achieve substantially higher performance by capturing nonlinear effects and interactions, with gradient boosting slightly outperforming random forest on test R^2 and cross-validation score.

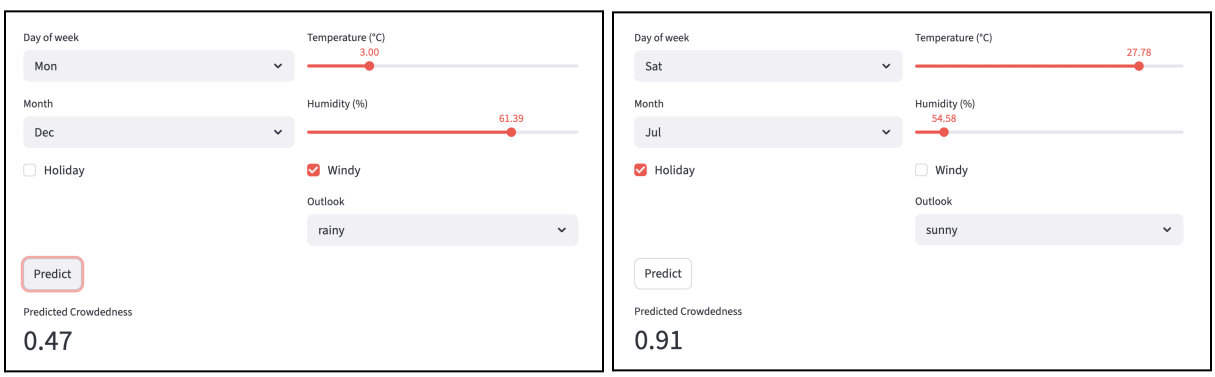
Results

Our deliverable includes a web application, developed using Streamlit, to visualize our results, and make them accessible and interpretable. The interface is organized into three components:

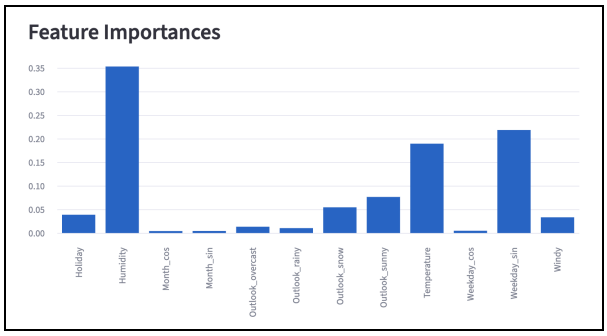


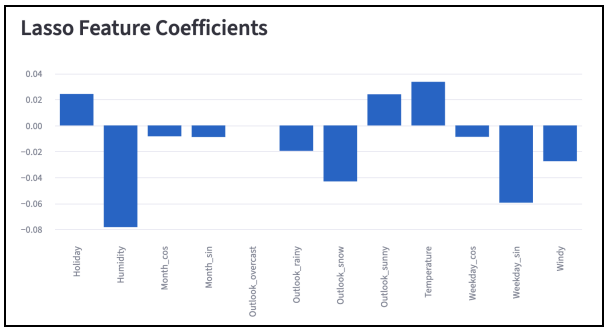
At the top, a model selection panel allows the user to switch between all four trained models, and displays a side-by-side accuracy comparison table showing the given components across all models simultaneously.

The input panel accepts the variables the course operator would realistically know in advance: day of the week, month, whether the day is a holiday, expected temperature, humidity, wind conditions, and weather outlook. Temperature and humidity sliders are dynamically bounded by the observed ranges for the selected month in the training data, preventing the user from entering physically implausible combinations. As shown from the images below, the different weather variables severely affect the crowdedness prediction.



At the bottom, the app renders a feature importance bar chart for the tree-based modes (left) and a coefficient chart for the linear models (right).





Conclusion

This project developed and compared four statistical learning models to predict golf course demand as measured by crowdedness. Linear regression and Lasso regression both showed the same test result of 0.58, which establishes the performance ceiling for additive models. Random forest and gradient boosting substantially outperformed both the previous, with gradient boosting achieving the best overall result at a test of 0.898 and cross-validated R^2 of 0.874. This gap in performance confirms that the drivers of demand interact in nonlinear ways that linear models cannot capture.

Across both tree-based models, three features consistently emerged as primary predictors of crowdedness: humidity, weekday, and temperature. This aligns with golfer intuition. Holiday status and weather outlook also contributed meaningfully, while cyclical month encoding captured broader seasonal trends.

As previously mentioned, the dataset is synthetically generated, constructed to resemble conditions in Sendai, Japan, a climate and golf market that may differ significantly from courses in other regions. Despite these limitations, the modeling framework can be somewhat viable. Given real historical booking data from a specific course, the same preprocessing pipeline and ensemble methods could be applied to build a demand forecasting tool grounded in actual golfer behaviour.

Dynamic Pricing

Worcester Polytechnic Institute

Golf Course Demand Prediction and Dynamic Pricing

A machine learning approach to revenue optimization

Neel Awsare

MA4635: Data Analytics & Statistical Learning

Graduate Credit Extension

Professor Oren Mangoubi

05/01/2026

Introduction

Golf courses often use simple pricing patterns, such as charging more on weekends and less on weekday afternoons. While this captures broad demand trends, it does not adjust for daily differences in weather, season or holidays that all lead to varying course demand. This project extends the original golf course demand prediction model into a dynamic pricing system. Instead of only predicting how crowded the course will be, the goal is to set a daily price that maximizes expected revenue.

Theory

Dynamic pricing adjusts prices based on expected demand and customer willingness to pay. It is commonly used in industries with limited capacity, such as airlines, hotels, and event ticketing. Golf courses fit this structure because tee times are limited and unused capacity cannot be recovered after the timeslot passes.

The main concept behind dynamic pricing is price elasticity of demand. If customers are highly price sensitive, demand decreases sharply when price increases. If customers are less price sensitive, prices can increase with only a small drop in demand. In this project, demand is adjusted using an exponential elasticity formula:

$$D(P) = D_0 e^{-\beta(P-P_0)}$$

where D_0 is baseline demand, P is price, P_0 is the reference price, and β controls customer sensitivity to price. A reference price is used so that demand remains close to baseline at a normal price level. Prices above the reference price reduce demand, while prices below it increase demand.

Since the dataset has no real price variable, price sensitivity is simulated. Beta is programmed to adjust based on course conditions: good weather, holidays, and high baseline demand make customers less price sensitive, while poor weather and low baseline demand make them more price sensitive. Random noise is also added to better reflect unpredictable customer behavior.

Models

The project uses a three-part modeling pipeline. First, a baseline random forest model predicts crowdedness from weather and calendar variables, including temperature, humidity, wind, outlook, season, holiday status, and weekday. Random forest was chosen because it can capture nonlinear relationships well between course conditions and demand.

Second, a simulated pricing dataset is created. A price between $20 and $40 is assigned to each observation. The baseline demand prediction is then adjusted using the elasticity formula to create a simulated price-adjusted demand target. Beta is used only to generate this target and is not included as a feature in the second model.

Third, an additional machine learning model predicts adjusted demand using course conditions, baseline demand, and price. This model estimates:

$$D_{adj} = g(X, D_0, P)$$

where X represents course conditions, D_0 is baseline demand, and P is price. Random forest and gradient boosting are both candidates for this model, because the nonlinear relationships between course conditions and demand need to be captured again.

Finally, grid search is used for optimization. For each day, the model tests many prices between $20 and $40, predicts demand at each price, calculates revenue, and selects the price with the highest expected revenue:

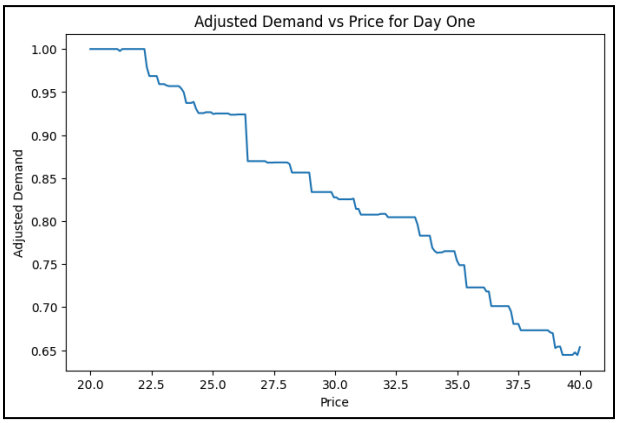
$$Revenue = D_{adj} \times P \times Capacity$$

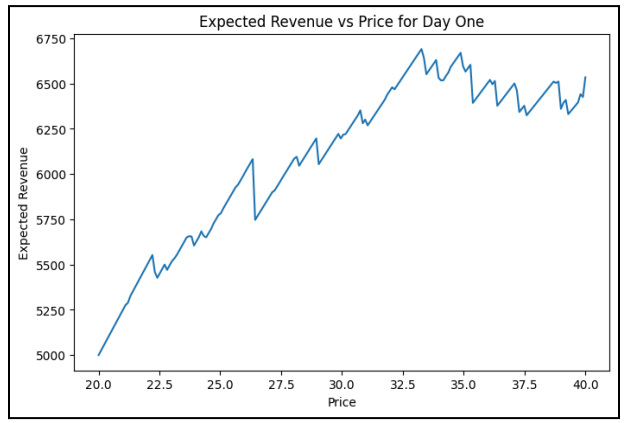
For model realism, a sensitivity analysis was performed to determine a realistic value for base beta (before scalar modifications based on course conditions). The following results were produced, and 0.0275 was chosen for beta to align the mean optimal price with the middle of the price range.

Base Beta	Mean of Optimal Price	Std Dev of Optimal Price
0.01	40.0	0
0.02	37.77	3.95
0.0275	30.56	5.39
0.05	24.11	1.92

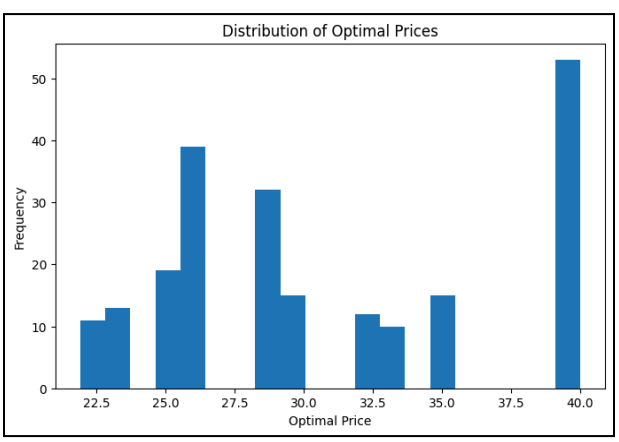
Results

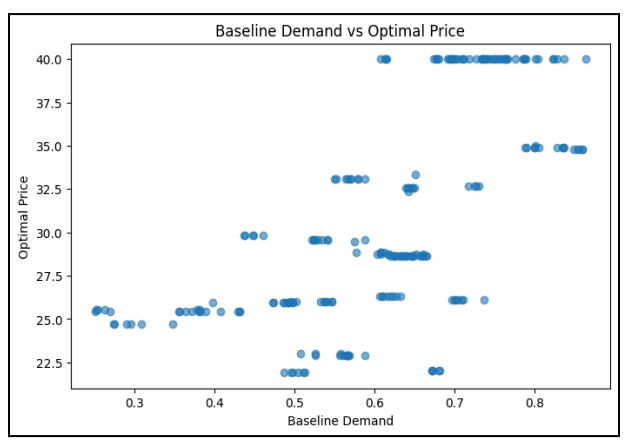
The baseline demand model achieved an R^2 of 0.853 and an $RMSE$ of 0.059. Since crowdedness ranges from 0 to 1, this means the typical prediction error is 5.9%. For the second price-demand model, random forest achieved an R^2 of 0.971 and an $RMSE$ of 0.031, and gradient boosting achieved an R^2 of 0.976 and an $RMSE$ of 0.028. This strong performance is expected because the adjusted demand target was simulated from a structured elasticity formula. Though performance was similar, gradient boosting was chosen as the stronger model for price-demand. The plots below show an example of the price-demand relationship for day one and the resulting revenue.





Across 1,095 days, dynamic pricing produced $1,064,914 in total revenue, compared to $997,241 from fixed pricing. This is a gain of $67,673, or 6.7%, which is a realistic improvement for a dynamic pricing model. Instead of uniformly raising prices, the model accounts for the negative relationship between price and baseline demand, resulting in a well-distributed set of optimal prices across different conditions.





Conclusion

This project shows how a demand prediction model can be extended into a dynamic pricing system. The original model predicts crowdedness from course conditions, while the extension uses those predictions to simulate price-sensitive demand and recommend revenue-maximizing prices.

The results suggest that dynamic pricing can create meaningful revenue gains with little to no extra costs, without requiring extreme price changes or drastic changes in customer demand habits. A 6.9% improvement is realistic for this application and shows the value of using machine learning for pricing decisions.

However, the results depend on several assumptions. The dataset is synthetic, does not include real price data, and does not explicitly define crowdedness. Therefore, the pricing model should be viewed as a proof of concept rather than a fully deployable system. With real booking and pricing data, this framework could be improved and used for more reliable golf course revenue optimization.